Week9_예습과제_진예원

10. 비전 트랜스포머

10.1 ViT

10.1.1 합성곱 모델과 ViT 모델 비교

10.1.2 ViT의 귀납적 편향

10.1.3 ViT 모델

10.1.4 패치 임베딩

10.1.5 인코더 계층

10.1.6 모델 실습

1. FashionMNIST 데이터세트

2. 이미지 전처리

3. **ViT 데이터로더 적용**

4. 사전 학습된 ViT 모델

5. 패치 임베딩 확인

6. 하이퍼파라미터 설정

7. 매크로 평균 F1 점수

8. ViT 모델 학습

9. ViT 모델 성능 평가

10.2 Swin Transformer

10.2.1 ViT와 스윈 트랜스포머 차이

10.2.2 스윈 트랜스포머 모델 구조

패치 파티션 / 패티 병합

스윈 트랜스포머 블록

10.2.3 모델 실습

1. 사전 학습된 스윈 트랜스포머 모델

2. 패치 임베딩 모듈

3. 스윈 트랜스포머 블록

4. SwinLayer 구조

5. W-MSA, SW-MSA 모듈

6. 패치 병합
7. 스윈 트랜스포머 모델 학습
10.3 CvT
10.3.1 합성곱 토큰 임베딩
10.3.2 어텐션에 대한 합성곱 임베딩
10.3.3 모델 실습
1. CvT 모델 이미지 데이터 전처리
2. 사전 학습된 CvT 모델
3. CvT 모델의 스테이지 구조
4. 셀프 어텐션 적용
5. CvT 모델 학습

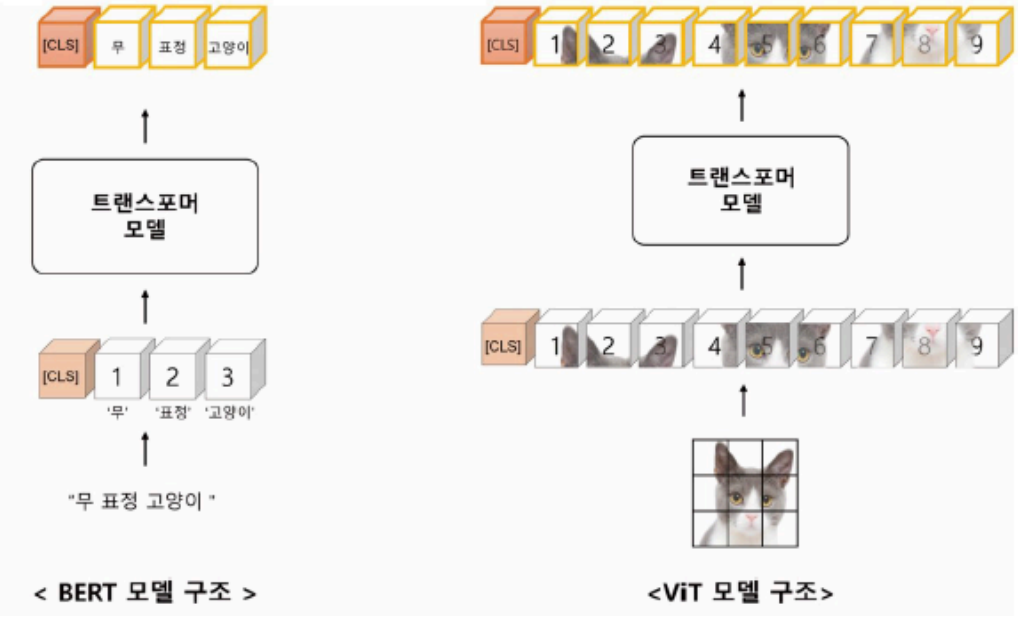
10. 비전 트랜스포머

1. **ViT(Vision Transformer)**
   - 2020년 구글에서 발표한 이미지 인식 딥러닝 모델
   - 자연어 처리에서 사용하는 Transformer의 Self-Attention을 이미지 분류에 적용
   - 기존 CNN과 달리 합성곱 계층을 사용하지 않고 전체 이미지를 한 번에 처리하는 방식으로 구현
   - 이미지 패치가 순차적으로 입력되어 이미지의 2차원 공간적 특성을 완전히 반영하기 어려움

     → 이러한 문제를 해결하기 위해 Swin Transformer, CvT 등장
2. **Swin Transformer**
   - Local Window를 이용해 각 계층의 Attention을 적용하는 패치의 크기와 개수를 다양하게 구성해 이미지 특징을 학습
   - 로컬 윈도우 내부와 윈도우 사이의 Attention을 모두 활용하여 이미지 특징 게층적으로 학습
   - 상대적 위치 편향을 반영하여 Attention 값 자체에 위치 정보 포함
3. **CvT(Convolutional Vision Transformer)**
   - 기존 합성곱 연산 과정을 ViT에 적용한 모델
   - 저수준 특징과 고수준 특징을 계층적으로 반영

    - 저수준 특징: 눈, 코, 입과 같은 작은 단위
    - 고수준 특징: 눈, 코, 입을 포함한 전체 얼굴
- Q(Query), K(Key), V(Value) 중 K와 V를 특징 벡터보다 축소해 계산 복잡도 감소

10.1 ViT



- **BERT와 ViT의 차이**
    - 두 모델 모두 자연어 처리에 널리 사용됨
    - 차이점: 입력 데이터를 만드는 과정
    1. BERT: 단어가 문장보다 작은 단위의 토큰들이 순차적으로 입력
    2. ViT: 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력 ⇒ 입력 이미지 패치는 왼쪽 → 오른쪽, 위 → 아래로 표현된 시퀀셜 배열을 가정

10.1.1 합성곱 모델과 ViT 모델 비교

- 두 모델 모두 이미지 특징을 잘 표현하는 임베딩을 만들고자 하는 목적은 같지만 과정에 차이가 있음

1. **합성곱 신경망(CNN)**
    - 이미지 패치 중 일부만 선택하여 학습 → 이미지 전체의 특징 추출

- ex) 고양이의 오른쪽 눈 표현하는 데 1,2,4,5만 학습 나머지 3,6,7,8,9 이미지 패치는 학습에 관여 X
- 좁은 수용 영역(Receptive Field, RF)을 가지므로 전체 이미지 정보 표현하는 데 수많은 계층 필요

2. **ViT**
   - 이미지를 작은 패치들로 나눠 각 패치 간의 상관관계를 학습 ← Self Attention 이용해 모든 이미지 패치가 서로에게 주는 영향을 고려해 이미지의 전체 특징 추출
   - 모든 이미지 패치가 학습에 관여하며 높은 수준의 이미지 표현을 제공할 수 있음
   - ex) Self Attention으로 모든 이미지 패치가 학습에 관여
   - (+) Attention Distance를 계산하여 오직 한 개의 ViT 레이어로 전체 이미지 정보 쉽게 표현 가능
   - (+) 픽셀 단위로 처리(CNN)하지 않고 패치 단위로 이미지를 처리 → 더 작은 모델로도 높은 성능 가능
   - (-) 입력 이미지 크기가 고정 → 크기가 다른 이미지 처리하려면 이미지 크기 맞추는 전처리 필요
   - (-) 이미지의 공간적인 위치 정보 고려(CNN)하는 것과 달리 패치 간의 상대적인 위치 정보만 고려 → 이미지 변환에 취약

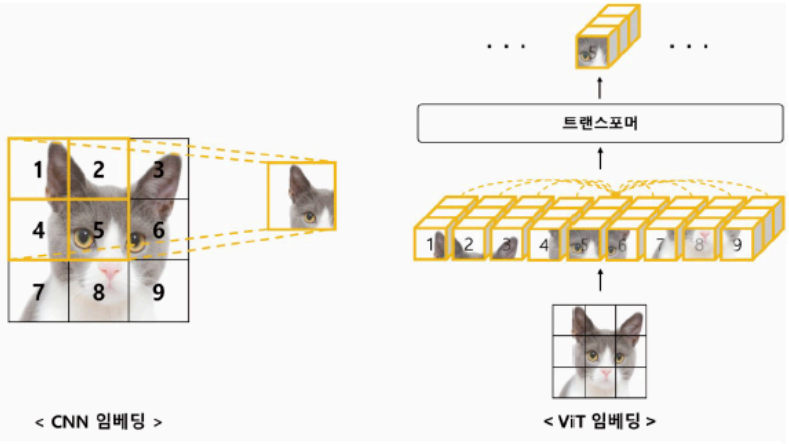


그림 10.3 합성곱 신경망과 ViT 계층의 차이

10.1.2 ViT의 귀납적 편향

- **귀납적 편향(Inductive Bias)**: 딥러닝 모델이 일반화 성능 향상을 위해 미리 가지고 있는 가정(Assumption)

모델별 귀납적 편향

1. **CNN - 지역적 편향(Local Bias)**
    - 이미지의 작은 영역에서 특징을 추출한 뒤 조합하여 전체 이미지 특징 추론
    - 지역적인 특징을 강조하는 구조
    - 이미지 데이터에서 좋은 성능 발휘
2. **RNN - 순차적 편향(Sequential Bias)**
    - 이전 시간의 상태를 기억하고 현재 입력과 결합하여 다음 상태 예측
    - 순차적인 특징을 강조하는 구조
    - 시계열 데이터에서 좋은 성능 발휘
3. **ViT - 귀납적 편향이 거의 없음**
    - 입력 데이터의 다양한 쿼리(Query), 값(Key), 값(Value)의 임베딩을 통해 일반화된 관계를 학습 → 귀남적 편향 거의 X
    - 대용량 데이터에서 이미지 특징 간 관계를 학습하는 데 유리

    ∴ 특정 관계 편향(귀납적 편향)이 강할수록 특정 데이터 구조를 학습하는 데 유리하지만, 다른 다양한 관계 표현하기 어려워 귀납적 편향이 약한 모델을 선호하게 됨

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

10.1.3 ViT 모델

- 패치 임베딩(Patch Embedding)과 인코더(Encoder) 계층으로 구성됨

1. **Patch Embedding**
    - 입력 이미지를 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환
2. **Encoder**
    - 각 패치와의 관계를 학습

- 전체 구조: 입력 이미지 특징 추출 → (분류나 회귀) 작업에 맞는 출력값으로 변환해 사용

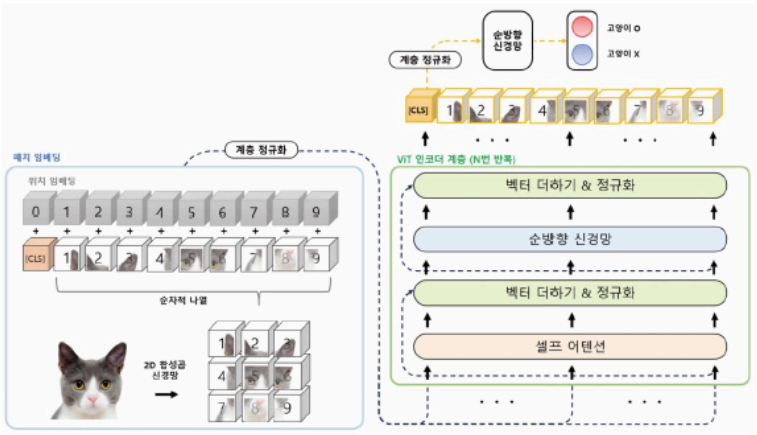


그림 10.4 ViT 모델 구조

10.1.4 패치 임베딩

- **Patch Embedding**: 입력 이미지를 작은 패치로 분할하는 과정

1. **이미지 패치 생성**
   - 입력 이미지를 패치로 분할하기 전에 일정한 크기로 조정하는 전처리 필요
     - ex) 640 × 480 이미지 → 224 x 224 정방향 이미지
   - 이미지 크기 조정한 후 합성곱 신경망의 계층을 활용하여 작은 패치로 분할 ← 패치 분할 시 Kernel Size와 Stride 설정 필요
   - **Kernel Size:** 패치의 크기 / **Stride:** 패치가 이동하는 폭 (두 값은 하이퍼파라미터로 정의)
     - $patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9-3}{3} + 1 = 3$

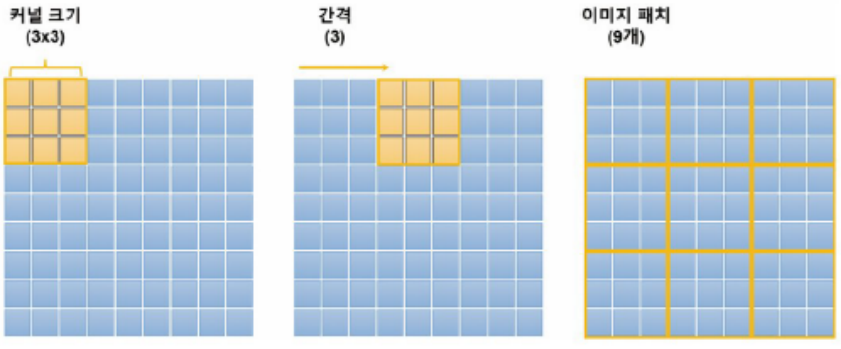


그림 10.5 이미지 패치 생성 과정

2. **분류 토큰(Special Classification Token, [CLS])**
   - 분할된 이미지 패치를 왼쪽 → 오른쪽, 위 → 아래 순서로 배열
   - 배열의 가장 왼쪽에 분류 토큰 추가

- 전체 이미지를 대표하는 벡터로, 특정 문제를 예측하는 데 사용

3. **위치 임베딩(Position Embedding)**
   - 인접한 패치 간 관계를 학습하기 위해 사용
   - 패치의 위치 정보를 임베딩 벡터로 변환하고 기존 이미지 패치 벡터들과 더함
   - 마지막으로 계층 정규화(Layer Normalization)을 적용하여 패치 임베딩 완성

   ∴ 입력 이미지를 작은 패치로 나누고 위치 정보를 추가하여 Transformer가 처리할 수 있도록 변환하면 GPU 메모리의 한계 극복하고 더 큰 이미지 처리 가능

10.1.5 인코더 계층

- **Encoder Layer:** 패치 임베딩을 입력받아 이미지 패치 사이의 관계를 학습하는 계층
- **인코더 학습 과정**
  1. 이미지를 일정한 크기의 패치로 나눠 각 패치를 벡터로 변환
  2. 인코더 계층에 입력으로 전달해 다양한 Query, Key, Value 임베딩의 관계를 학습
  3. N개의 Encoder Layer를 반복적으로 적용
  4. 마지막 계층에서 전체 이미지의 특징을 나타내는 분류 토큰 특징 벡터 추출

     (분류 토큰: 이미지 데이터를 잘 표현하는 특징 벡터. 이후 다양한 이미지 분류 및 검색 문제 등에 활용)
- **이미지 분류 과정**
  - ViT는 이미지 분류 시 Sequential Output(시퀀셜 산출물)을 모두 사용하는 것이 X 전체 이미지의 특징을 잘 표현하는 분류 토큰 벡터만 사용해서 분류 수행
  - 분류 토큰 벡터를 순방향 신경망(완전 연결 계층)에 연결하고 분류
  - ex) 고양이인지 아닌지 분류

10.1.6 모델 실습

- 이번 실습: 허깅 페이스 라이브버리와 FashionMNIST 데이터세트을 이용해 사전 학습된 ViT 모델을 미세 조정한다

1. FashionMNIST 데이터세트

- 기존의 MNIST 데이터세트보다 더 복잡한 이미지 분류 문제를 해결하기 위해 만들어짐
- 의류 이미지 담고 있는 데이터세트

- 총 10개 클래스
- 60000개 훈련 데이터세트 / 10000개 테스트 데이터세트

주요 코드 및 함수

- `torchvision.dataset.FashionMNIST` : 데이터세트 다운로드 및 불러오기
  - `root` : 데이터세트가 저장될 디렉터리 위치
  - `download=True` : 데이터세트가 없을 경우 다운로드
  - `train=True/False` : 훈련용 또는 헤스트용 데이터세트 선택
- `classes` : 데이터세트의 클래스 확인
  - `class_to_idx` : 클래스 ID와 클래스가 매핑된 값 확인
- `subset_sampler` : 서브 샘플링. 데이터세트(`dataset`), 클래스 목록(`classes`), 클래스별 최대 샘플링 개수(`max_len`)을 전달
- `defalutdict` : 기본값 있는 딕셔너리 클래스. 1000개의 데이터 색인 값 가져옴
  - 색인 값을 1차원 리스트로 풀어 `Subset` 클래스에 전달하면 해당 색인 값에 해당하는 데이터세트 추출 가능

출력 결과

- Training Data Size: 10,000
- Testing Data Size: 1,000
- 개별 데이터: PIL 이미지와 클래스 번호로 구성

→ 모델에 입력하기 위해 PIL.Image 형식이 아닌 Tensor 형식으로 변환하는 전처리 필요

2. 이미지 전처리

- **AutoImageProcessor**(이미지 프로세서 클래스): 사전 학습된 ViT 모델을 활용하여 이미지 전처리 수행
  - 자르기(crop)이나 크기 조정(resize) 등의 전처리 수행 가능
  - 실습에서는 google/vit-base-patch16-224-in21k 모델 사용
    - ImageNet-21k 데이터세트로 사전 학습
    - 입력 이미지 크기: 224 x 224
    - 패치 크기: 16

전처리 과정

1. `transforms.ToTensor()` : PIL 이미지 → Tensor 변환
2. `transforms.Resize()` : ViT 입력 크기인 224 x 224로 조정
3. `transforms.Lambda()` : 단일 채널 이미지를 복제하여 다중 채널 이미지(3채널)로 변환
4. `transforms.Normalize()` : 평균과 표준편차를 이용해 정규화

3. ViT 데이터로더 적용

- 이미지 전처리가 완료되어도 ViT 모델은 Tensor 형식이 아닌 딕셔너리 형식의 입력을 받음
- 입력 구조: `{"pixel_values": pixel_values, "labels": labels}`

  → 데이터로더에 `collcate_fn` (집합 함수)를 적용하여 미니 배치의 샘플 목록을 병합. ViT 모델 구조에 맞는 형태로 변환해 데이터로더에 적용

주요 코드 및 함수

- `collator()` : 사용자 정의 함수. transform 변수를 함께 전달해 이미지 변환하고 딕셔너리 구조로 값을 반환함.
  - 입력값 - 데이터로더에서 불러온 배치
  - 이번 실습에서는 ViT 모델에 맞는 입력 구조 변환 수행
  - `pixel_values` : (배치크기, 채널 수, 이미지 높이, 이미지 높이)로 구성됨
  - `labels`: 클래스 색인 값 반환
- `collate_fn` : 데이터로더에서 불러온 배치에 집합 함수 적용

출력 결과

- `pixel_values` : (배치 크기, 채널 수, 이미지 높이, 이미지 너비)
- `labels` : 배치에 포함된 이미지의 클래스 인덱스

  → 데이터로더 통해 ViT 모델의 입력 구조에 맞게 이미지와 레이블을 변환

4. 사전 학습된 ViT 모델

- 데이터로더 구성 완료 → 사전 학습된 ViT 모델을 불러옴
- **ViTForImageClassification** 이용하여 사전 학습된 모델 불러옴
  - 허깅 페이스 라이브러리에서 제공하는 ViT 이미지 분류 모델 클래스

- 21,841개의 분류 카테고리로 학습된 모델이므로 미세 조정 필요 ← 매개변수 통해 미세 조정 가능

모델 미세 조정

- **fine-tuning:** 사전 학습된 모델을 새로운 데이터세트와 작업에 맞게 추가 학습시키는 과정
- `num_labels` : 레이블 개수, `id2label` : ID/레이블, `label2id` : 레이블/ID 매개변수 통해 현재 데이터 세트에 적합한 구조로 모델 미세조정
- 출력 결과: 분류기(`classifier`)의 출력 데이터 차원 크기(`out_features`)를 10개로 반환

∴ 기존 사전 학습된 ViT의 특징 추출 구조를 활용하되, 최종 분류기는 현재 데이터셋에 맞게 변경

5. 패치 임베딩 확인

- ViT는 기존 Transformer와 유사하지만, 입력 데이터를 일정한 크기의 패치로 분할한 후 패치마다 특성을 추출하는 과정인 **Patch Embedding**을 추가하여 수행

패치 임베딩의 처리 과정

1. 패치 임베딩 처리 방식 확인 위해 배치 데이터 하나를 **패치 임베딩 함수 (model.vit.embeddings.patch_embeddings)**에 적용해봄
   - 16 × 16 커널 & 16 간격의 합성곱(Conv2d) 연산 수행
   - ex) 224 × 224 크기의 이미지에 합성곱 연산 수행시: $\frac{224-4}{4}+1=14$이므로 14 x 14 텐서 생성 → 일렬로 나열하면 196(14x14)개 패치 생성
2. **임베딩 함수(model.vit.embeddings)** 수행
   - [4, 1, 768] 크기의 [CLS] 토큰이 패치 앞에 붙어 197개의 벡터 생성됨

∴ Patch Embedding은 이미지를 196개 패치로 변환하고, 분류 토큰을 추가해 197개의 벡터를 생성하는 과정

6. 하이퍼파라미터 설정

- **Training Arguments(학습 매개변수 클래스)**: 모델 학습에 필요한 다양한 인자들을 저장하고 관리
  - 하이퍼파라미터 관리
  - 모델 평가

- 분산 학습
- 허깅 페이스 허브 연동
- 로그 추정
- 80여 개의 매개변수 통해 다양한 기능 설정 가능

표 10.2 학습 매개변수 클래스(TrainingArguments)

매개변수	의미
output_dir	체크포인트 저장 경로
save_strategy	체크포인트 저장 간격 설정 ▪ no: 저장 안 함 ▪ steps: 스탭마다 ▪ epoch: 에폭마다
save_steps	save_strategy= "steps"일 때 스탭 간격 설정
save_total_limit	체크포인트 최대 저장 개수
evaluation_strategy	체크포인트 평가 간격 설정 ▪ no: 저장 안 함 ▪ steps: 스탭마다 ▪ epoch: 에폭마다

매개변수	의미
learning_rate	초기 학습률
per_device_train_batch_size	학습 배치 크기
per_device_eval_batch_size	평가 배치 크기
num_train_epochs	학습 반복 수
weight_decay	가중치 감쇠
load_best_model_at_end	모델 불러오기 시 최상의 모델 선택 여부
metric_for_best_model	최상의 모델 선정 기준이 되는 평가 방식 설정 ▪ accuracy: 정확도 ▪ precision: 정밀도 ▪ f1: F1 점수 ▪ mae: 평균 절대 오차 ▪ mse: 평균 제곱 오차 ▪ rmse: 평균 제곱근 편차
logging_dir	로그 저장 폴더
logging_steps	로그 출력 간격
seed	학습 시작 시 설정되는 무작위 시드
fp16[5]	Mixed precision 사용 여부

7. 매크로 평균 F1 점수

- 하이퍼파라미터 설정이 완료되면 모델의 성능을 평가할 지표를 설정함
- 평가 지표: **Macro Average F1-Score(매크로 평균 F1 점수)**

    - 이번 실습은 10개 클래스를 분류하는 다중 클래스 분류 문제

      → 모든 클래스에 대한 F1 점수의 가중 평균으로 전체 모델의 성능 측정

- 성능 평가 지표도 허깅 페이스의 평가 라이브러리로 메트릭을 쉽게 불러올 수 O
    - **메트릭 불러오기 함수(load):** 정확도(accuracy), 정밀도(precision), F1 점수(f1) 등 다양한 평가 지표 제공
    - 이 함수 통해 F1 점수 방식 불러옴

성능 평가 과정

1. `compute_metrics()` 함수의 `eval_pred` 인수 통해 모델이 예측한 결괏값 전달받음
    - 모든 클래스에 대한 예측값 포함되어 있으므로, 가장 높은 값을 선택해 `predictions` 변수에 저장
2. 예측 데이터를 정제했다면, 예측값(`predictions`)과 실젯값(`labels`)을 메트릭 연산(`metric.compute`) 메서드에 전달
    - 매크로 평균 F1 점수 방식 사용 → 평균(`average`) 매개변수에 `macro` 를 입력

8. ViT 모델 학습

- 사전 학습된 ViT 모델, 데이터로더, 학습 매개변수, 평가 지표의 설정이 완료되면 실제 모델 학습을 진행
- **Trainer 클래스(훈련자 클래스)**
    - 트랜스포머스 라이브러리에서 제공하는 훈련자 클래스
    - 모델을 함수(`model_init`) 형태로 생성하여 전달
    - 데이터 로더를 내부적으로 수행하므로 데이터세트와 집합 함수를 함께 전달
    - `tokenizer` 에는 `image_processor` 전달

학습 과정

1. 학습에 필요한 설정 완료 후 `trainer.train()` 으로 학습 진행
2. 학습 완료 시 `ViT-FashionMNIST` 폴더에 체크포인트별 학습 결과, 성능 지표, 하이퍼파라미터 등이 저장됨

최종 학습 결과

표 10.3 ViT 학습 결과

Epoch	Training Loss	Validation Loss	F1-Score
1	0.7062	0.6377	0.8905
2	0.4954	0.4781	0.9166
3	0.4078	0.4357	0.9231

- 손실값: 0.4078
- F1 점수: 0.9231

  → 학습이 우수하게 진행되었음을 확인

체크포인트 불러오기

- ViT-FashionMNIST 폴더에 저장된 체크포인트를 불러올 때 `model_init` 함수의 `model` 변수에서 `pretrained_model_name_or_path` 매개변수를 설정
- ex) `../model/ViT-FashionMNIST/checkpoint-1875`

9. ViT 모델 성능 평가

- **혼돈 행렬(Confusion Matrix)**
  - 테스트 데이터세트를 통해 성능 지표 확인
  - 혼돈 행렬로 클래스별 예측 결과 확인

실습 결과

- 테스트 데이터세트에 대한 분류 정확도: 약 92% → 정확도 측면에서 중수한 성능
- but 혼돈 행렬로 클래스별 성능 지표 확인해보면 셔츠를 티셔츠나 코트로 오분류한 경우가 상대적으로 많음

모델 개선 방법

1. 하이퍼파라미터 조정
2. 모델 구조 변경
3. 전처리 과정 개선
4. 데이터 증강

→ 모델 성능 개선하고, 혼돈 행렬로 오분류가 발생하는 클래스의 문제점을 해결해 더 높은 정확도를 달성할 수 있음

10.2 Swin Transformer

- **스윈 트랜스포머:** 2021년에 발표된 대규모 비전 인식 모델
- **기존 transformer의 문제점**
  1. 고정된 패치 크기
     - 세밀한 예측이 필요한 semantic segmentation(의미론적 분할) 등에 적합하지 X
  2. 높은 계산 복잡도
     - Transformer의 Self-Attention은 입력 이미지 크기에 대한 2차(Quadratic) 계산 복잡도를 가짐
     - 고해상도 이미지 처리에 어려움
- **Swin Transformer의 특징**
  - 계층적(hierarchy) 특징 맵 구성해 1차(Linear) 계산 복잡도
  - 이미지 분류, 객체 감지, 영상 분할 등 인식 작업에서 강력한 성능
  - **shifted window(시프트 윈도)** 기술 사용
    - 입력 이미지를 일정한 크기의 패치로 분할하여 윈도 구성
    - 구성된 윈도 영역 내에서만 Self-Attention 수행
    - 패치를 겹쳐 더 큰 효율성 제공
  - 계층마다 어텐션이 수행되는 패치의 크기와 개수를 다르게 적용해 처리

  → 다양한 크기의 객체를 효율적으로 처리할 수 있음
- 객체 탐지나 객체 분할 등과 같은 작업에서 백본 모델로 사용됨

10.2.1 ViT와 스윈 트랜스포머 차이

- **ViT의 한계**
  - 획일적인 패치 크기와 위치에 대한 제약 → 다양한 이미지 크기와 종횡비를 가진 데이터세트에 대해서는 적용 어려움
  - 패치 단위로 입력 받음 → 이미지의 공간 정보가 완전히 보존되지 않을 수 있음
  - 패치 수 증가 → 학습 데이터 크기도 증가 → 모델 학습에 필요한 계산량 증가. 이로 인해 대규모 데이터세트에서는 학습에 필요한 컴퓨팅 자원 많이 사용
- **Swin Transformer의 개선 방법**

1. **Local Window(로컬 윈도)**
    - 입력 이미지 처리하기 위해 사용되는 고정 크기의 작은 윈도
    - 물체의 크기나 해상도를 계층적으로 학습
2. **Shifted Window(시프트 윈도)**
    - 로컬 윈도를 이동시켜 다음 계층에 사용
    - 패치 위치를 더 유연하게 다루고 인접 패치 간 정보 계산

→ ViT보다 적은 매개변수로 더 넓은 범위의 이미지 크기와 종횡비를 다룰 수 있음

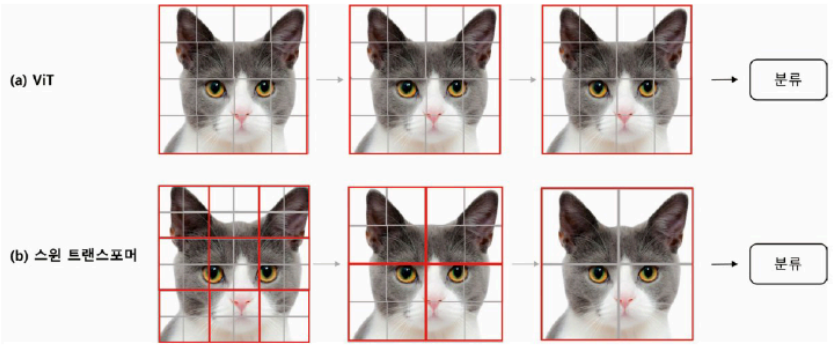


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

- 그림 10.6 네트워크 구조 비교
    - 회색 격자: 패치
    - 빨간색 격자: 로컬 윈도
    - ViT: 계층마다 전체 이미지 안에 어텐션이 수행되는 패치 크기와 개수가 동일
    - Swin Transformer: 로컬 윈도 통해 계층마다 어텐션이 수행되는 패치 크기와 개수를 다양하게 적용

∴ 로컬 윈도로 패치의 세부 정보를 학습하고, 시프트 윈도로 전역 특징을 확인하여 고해상도 이미지를 효율적으로 처리

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

10.2.2 스윈 트랜스포머 모델 구조

- **전체 구조:** 패치 파티션 → 선형 임베딩(Linear Embedding) → 스윈 트랜스포머 블록(Swin Transformer Block) → 패치 병합(Patch Merging)
  - 입력 이미지를 일정한 크기의 패치로 분할
  - 각 패치에 대해 선형 임베딩 수행 → 각 패치는 고정된 차원의 벡터로 변환됨
  - 분할된 패치들을 기반으로 스윈 트랜스포머 블록 구성
    - 이 블록은 어텐션 계층, 계층 정규화, 다층 퍼셉트론 등을 포함하며 패치 간 상호작용을 수행함
  - 패치를 병합해 전체 이미지에 대한 분류 수행. 순차적으로 합치면서 이미지의 전체적인 정보를 추출하여 분류

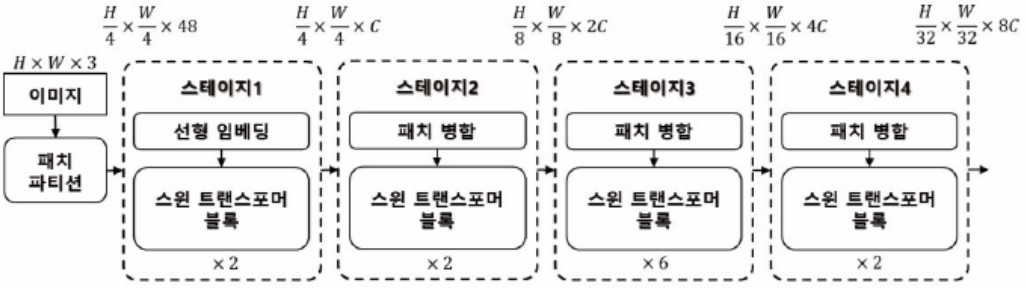


그림 10.7 스윈 트랜스포머 구조

- **스테이지 -** 총 4개 스테이지로 구성/각 스테이지를 서로 다른 해상도와 패치 크기를 가짐
  - 스테이지 1: 패치에 선형 임베딩 수행 → 스윈 트랜스포머 블록에 전달됨
  - 스테이지 2,3,4: 배치 병합 → 각 스윈 트랜스포머 블록에 전달됨
    - 패치 병합: 모델의 매개변수 줄이기 위해 이미지 텐서 [C, H, W] → [2C, H/2, W/2] 재정렬해 저차원 임베딩 수행
  - 공통 구조: 스윈 트랜스포머 블록은 윈도 격자 안에 있는 패치 내부의 관계, 윈도 격자 간의 관계를 학습하는 두 가지 어텐션 모듈로 구성

패치 파티션 / 패티 병합

- **패치 파티션(Patch Partition)**: 입력 이미지를 작은 사각형 패치로 분할하여 처리하는 방식
  - 분할된 패치는 트랜스포머 계층의 입력으로 사용됨

- 입력 이미지의 공간 정보를 보존
- 트랜스포머 계층에 대한 계산 효율성 향상
- 일반적인 이미지 텐서는 [C, H, W] 구조 → 패치 파티션 통해 로컬 윈도 추가
- 예시
  - 입력 텐서: [3, 32, 32]
  - 패치 크기: 4 x 4
  - 패치의 가로, 세로 크기가 4가 됨

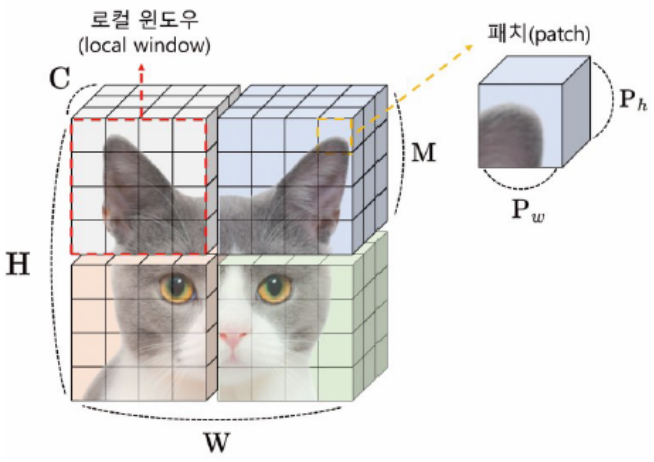


그림 10.8 패치 파티션 구조

- **패치 병합(Patch Merging):** 인접한 패치들의 정보를 저차원으로 축소하는 과정
  - 모델 매개변수 수 줄이기 위해 이미지 패치 텐서 [C, H, W]를 [2C, H/2, W/2]로 재정렬
  - 이후 2C 차원을 저차원으로 임베딩
  - 예시
    1. 입력 텐서 [3, 8, 8] → [6, 4, 4]로 재정렬
    2. 증가된 6차원 채널을 원래 3차원으로 임베딩
    3. 최종 텐서 [3, 4, 4] 생성

스윈 트랜스포머 블록

- 패치 파티션 이후 네 개의 스테이지 안에서 반복적으로 적용
- 선형 임베딩 또는 패치 병합 이후 수행

- 계층 정규화, W-MSA(Window Multi-head Self-Attention), MLP, SW-MSA(Shifted Window Multi-head Self-Attention) 등으로 구성

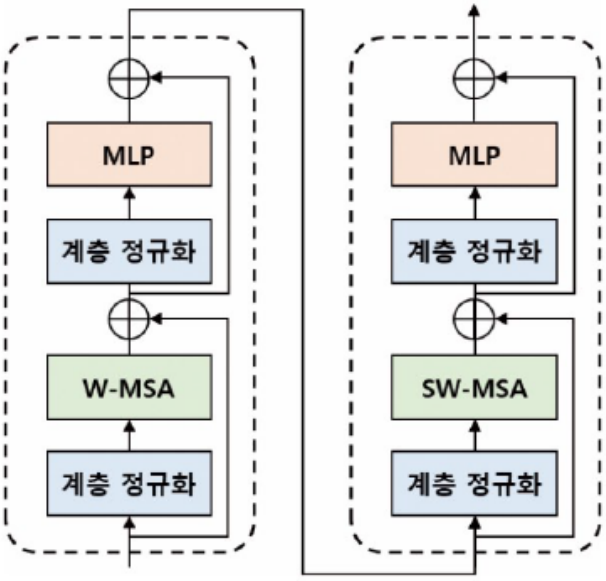


그림 10.9 스윈 트랜스포머 블록 구조

- 스윈 트랜스포머는 일반적인 MSA 대신 W-MSA와 SW-MSA를 순차적으로 수행함

1. **W-MSA(Window Multi-head Self-Attention)**
    - 로컬 윈도를 사용하는 멀티 헤드 셀프 어텐션
    - 입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도에서 독립적으로 셀프 어텐션을 수행함
    - 이미지에서 일정한 크기의 윈도 영역을 설정하고, 해당 영역 내 픽셀 간의 어텐션 계산
    - 이미지 내의 전체적인 어텐션 수행하고, 특정 위치에 대한 정보 추출
    - 각 윈도 내 지역적인 특징 분석할 수 있음
    - 전체 입력에 대한 셀프 어텐션 비용을 2차에서 1차 계산 복잡도로 줄임
    - 연산량 대폭 감소 but 윈도 내부 이미지 패치 영역에만 셀프 어텐션 수행하므로 윈도 사이의 연결성 약함
2. **SW-MSA(Shifted Window Multi-head Self-Attention)**
    - W-MSA의 한계를 보완하기 위해 사용
    - 윈도 사이의 연결성 구축해 윈도 간의 관계성 정보를 추출
    - 윈도를 가로 또는 세로 방향으로 이동한 후 인접한 윈도 간의 셀프 어텐션 계산

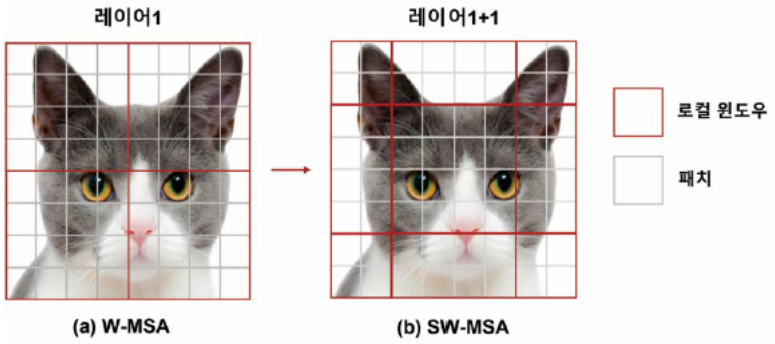


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- 그림 10.10에서의 W-MSA & SW-MSA
  1. W-MSA
     - 로컬 윈도(빨간색 영역) 내부에서 패치 간 셀프 어텐션 수행
     - 로컬 윈도가 이동하면서 각 윈도에서 연산 수행해야 하므로 계산량 많아지는 것처럼 보이지만
     - **효율적인 배치 계산(Efficient Batch Computation)** 통해 연산량 감소

       → 로컬 윈도에서 연산할 때도 배치 축을 활용하여 계산 속도 향상
  2. SW-MSA
     - 로컬 윈도 간의 셀프 어텐션 수행 위해 W-MSA에서 사용된 로컬 윈도 개수가 많으므로 더 많은 셀프 어텐션 연산 요구
     - 이를 해결하기 위해 **순환 시프트(Cyclic Shift)**와 **어텐션 마스트(Attention Mask)** 사용

     a. **순환 시프트(Cyclic Shift)**: 로컬 윈도가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져오는 방식
        - 로컬 윈도 크기의 절반(M/2)만큼 이동
        - 이동된 위치에서 연산 수행하지 않도록 어텐션 마스크 사용해 방지
          - 어텐션 마스크: 로컬 윈도가 이동한 위치에서 연산 수행하지 않게 하는 역할
        - 연산 후 **역순환 시프트(Reverse Cyclic Shift)**로 원래 로컬 윈도 구조 복원

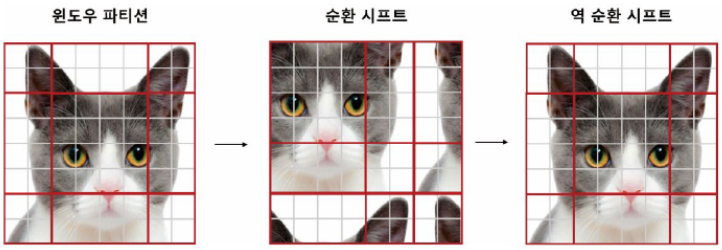


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

b. **어텐션 마스트(Attention Mask)**

- 순환 시프트 후 원래 인접하지 않은 영역끼리 연산하지 않도록 방지
- 로컬 윈도의 각 영역에서 연산할 수 있는 패치를 제한
- 순환 시프트 통해 이동된 윈도에 대한 정보 보존하면서도, 불필요한 연산 줄이고 효율적인 계산 가능하게 함

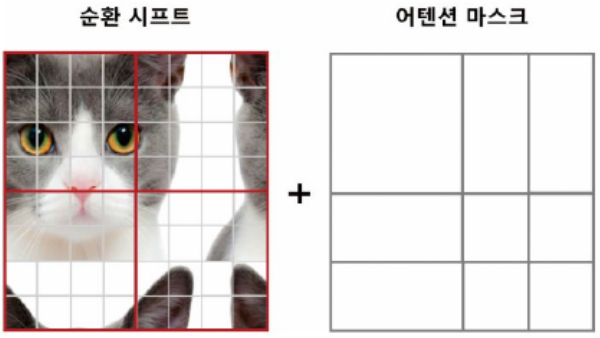


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

→ 순환 시프트와 어텐션 마스크를 함께 활용하여 로컬 윈도 간 셀프 어텐션을 효율적으로 수행

상대적 위치 편향(Relative Position Bias)

- 상대적 위치 편향: 로컬 윈도 내부에 있는 패치 사이의 상대적인 거리를 임베딩하는 방법
- 기존 ViT와 다른 스윈 트랜스포머의 특징
- 패치 간 상대적인 거리를 임베딩하는 목적

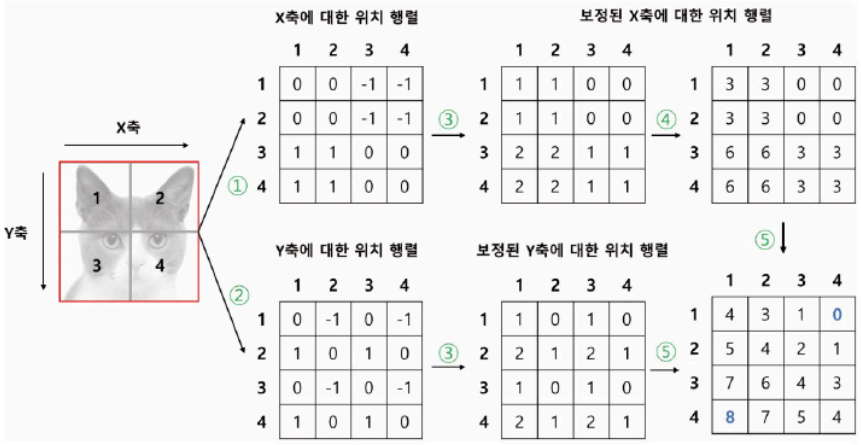


그림 10.13 상대적 위치 편향 계산 과정

- **상대적 위치 행렬**: 이미지 패치 1~4가 주어졌을 때 상대적 거리를 X축과 Y축의 두 방향으로 표현
  - X축: 가로 방향 거리
  - Y축: 세로 방향 거리
- 그림 10.13
  - 기준 패치와 같은 위치 → 상대적 거리 0
  - 오른쪽으로 1칸 → X축 상대적 거리 -1
  - 왼쪽으로 1칸 → X축 상대적 거리 +1
  - 아래쪽으로 1칸 → Y축 상대적 거리 -1
  - 위쪽으로 1칸 → Y축 상대적 거리 +1
- **예제 10.10 상대적 위치 편향 계산**
  - `window_size` : 여러 개의 패치를 포함하는 격자 크기. 윈도 내외부의 패치를 구분해 주는 역할
  - `torch.meshgrid()` : `coords_h` 배열 값(i)과 `coords_w` 배열 값(j)으로 사각형 격자(ij) 생성. 사각형의 X좌표, Y좌표에 해당하는 배열 반환
  - `torch.stack()` : (X,Y) 쌍 배열 생성
  - `torch.flatten()` : 각 X, Y축에 대한 위치 인덱스(`coords_flatten`) 생성

  → 각 위치 사이의 상대적 좌표 계산
- **예제 10.11 X, Y축에 대한 위치 행렬**
  - X축과 Y축의 상대적 위치 행렬을 각각 분리

- `relative_coords` 를 각 X, Y 좌표에 대한 `x_coords` , `y_coords` 로 분리한 후 각 `x_coords` , `y_coords` 에 대해서는 `window_size` -1만큼 더해줌
- `x_coords` 에 대해서 2 * window_size -1만큼 곱해줌
- `x_coords` 와 `y_coords` 를 더해주면 X, Y축에 대한 상대적 위치 좌표 행렬 생성 가능

- **예제 10.12 X, Y축에 대한 상대적 위치 좌표 변환**
  - `num_heads` : MSA를 계산할 때 사용하는 헤드 수
  - `relative_position_bias_table` : 헤드별로 0~8번까지 9개의 상대적 위치 편향을 가지는 테이블
  - `relative_position_index` : 각 위치의 편향 값을 불러오기 위한 인덱스
  - `view()` : 패치별 어텐션 계산에 사용할 텐서 형태로 재구성(헤드별 i번째 패치에서 j번째 패치의 어텐션 값 더해지는 텐서 [4,4,1]로 재구성)
- **상대적 위치 편향을 고려한 셀프 어텐션**

  $$Attention(Q, K, V) = SoftMax(QK^T/\sqrt{d} + B)V$$
  - 생성된 상대적 위치 임베딩을 기존 트랜스포머에 사용되는 셀프 어텐션 수식에 적용
  - Q: 쿼리 / K: 키 / V: 값
  - d: 벡터의 임베딩 차원
  - B: 상대적 위치 임베딩

10.2.3 모델 실습

- 이번 실습에서는 앞선 ViT 모델의 실습 코드를 활용해 Swin Transformer 모델을 미세조정함
- 사용 모델: `microsoft/swin-tiny-patch4-window7-224`
  - 허깅 페이스 라이브러리에서 제공하는 사전 학습된 모델
  - ImageNet-1k 데이터세트 사용
  - 입력 이미지 크기: 224 x 224
  - 패치 크기: 4 x 4
  - 로컬 윈도 크기: 7 x 7
- 앞선 ViT 실습에서 사용한 라이브러리와 코드 재사용

1. 사전 학습된 스윈 트랜스포머 모델

- 사전 학습된 스윈 트랜스포머 모델을 불러오고 모델 구조 확인
- **모델 구조**
  - 스윈 트랜스포머는 크게 `swin` 과 `classifier` 로 구분
  - `swin` : 패치 파티션, 선형 임베딩, 4개 스테이지(0~3)로 구성
  - 스테이지 이후 계층 정규화(`layernorm`)와 풀링(`pooler`)을 통과
  - `classifier` 에서 클래스 구분
- **패치 임베딩**
  - `model.swin.embeddings.patch_embeddings` 에서 수행
  - (`patch_embeddings` 에서 수행되는 합성곱 계층) 4 × 4 Kernel Size, Stride 4를 적용
  - 224 × 224 이미지 적용하면 $\frac{224-2}{4}+1=56$이 되어 56 x 56개의 텐서 생성
  - 이 텐서를 일렬로 나열하면 3,136개의 패치 생성

2. 패치 임베딩 모듈

- **출력 결과**
  - 이미지 차원: [32, 3, 224, 224]
  - 합성곱 계층: Conv2d(3, 96, kernel_size=4, stride=4)
  - 패치 임베딩 차원: [32, 3136, 96]

    → 3,136개의 패치 생성되고 96개의 채널 생성
- **ViT와의 차이**
  - ViT: 196개의 패치 + [CLS] 토큰 1개 = 197개 패치 생성
  - Swin Transformer: [CLS] 토큰 사용하지 않고 토큰들의 평균값을 사용해 분류 진행
  - 객체 검출같은 작업에서 입력 이미지에 여러 물체 포함되면 [CLS] 토큰 하나로는 모든 물체 효과적으로 검출하기 어려움

    → 평균값 채택해 모델의 성능 향상

3. 스윈 트랜스포머 블록

- 패치 임베딩 구조 확인 후 스윈 트랜스포머의 스테이지 구조 확인
- **스테이지 구조**
  - `blocks`: 스윈 트랜스포머 블록
  - `downsample`: 패치 병합 연산
- **SwinLayer**
  - `SwinLayer-0`: W-MSA 수행
  - `SwinLeyr-1`: SW-MSA 수행

4. SwinLayer 구조

- **SwinLayer의 구성:**
  1. `layernorm_before`: 첫 번째 계층 정규화
  2. `attention`: W-MSA 또는 SW-MSA
  3. `layernorm_after`: 두 번째 계층 정규화
  4. `intermediate`, `output`: MLP
  - 계층 정규화 → Attention → 계층 정규화 → MLP 순서로 수행
- **MLP 계층**
  - 두 개의 연속된 선형 변환 계층과 GELU 활성화 함수로 구성
  - **GELU(Gaussian Error Linear Unit)**
    - ReLU와 유사항 형태
    - 가우시안 분포에 대한 누적 분포 함수 사용
    - 입력값이 0 주변에서 부드럽게 변화

∴ 스윈 트랜스포머는 두 개의 연속된 선형 변환과 GELU 함수를 통해 입력 특징맵을 더 깊은 차원으로 매핑하고 ReLU를 사용했을 때보다 더 안정적으로 학습이 이뤄진다

5. W-MSA, SW-MSA 모듈

- **출력 결과**
  - 패치 임베딩 차원: [32, 3136, 96]
  - W-MSA 결과: [32, 3136, 96]

- SW-MSA 결과: [32, 3136, 96]

→ 세 텐서의 차원이 동일한 하이퍼파라미터로 구성됨

6. 패치 병합

- **패치 병합 과정**: 입력 패치를 절반 크기로 분할하는 `reduction`과 계층 정규화 수행
  - 3,136(56×56)개의 패치 → 784(28x28)로 분할
  - 분할된 패치는 네 개로 나뉘어 (N, 28, 28, 96)의 텐서가 생성됨
  - 이 네 개의 텐서를 채널에 병합하면 96x4 = 384가 됨
  - 이렇게 하면 채널이 줄어들지 않으므로 선형 변환 계층 통해 채널의 수를 절반(384 → 192)로 조정
- **출력 결과:** [N, 784, 192]
  - 같은 방식으로 총 4개 스테이지 진행
  - 스테이지 지날수록 패치 크기와 수는 많아지며 토큰의 차원은 두 배씩 증가
  - 마지막 스테이지를 통과하면 [N, 49(7×7), 768]이 됨
  - 이 특징맵이 풀링 계층 통과해 클래스 예측

7. 스윈 트랜스포머 모델 학습

- 전까지는 스윈 트랜스포머의 모델 구조 확인
- 이제 FashionMNIST 데이터세트를 활용해 스윈 트랜스포머 모델을 미세 조정
- `Trainer`를 통해 모델 학습 수행
- 학습 후 `Swin-FashionMNIST` 디렉터리에 체크포인트마다 학습 결과, 성능 지표, 하이퍼파라미터 저장

표 10.3 스윈 트랜스포머 학습 결과

Epoch	Training Loss	Validation Loss	F1
1	0.3720	0.3046	0.8985
2	0.2579	0.2688	0.9138
3	0.2607	0.2416	0.9159

- **결과 분석**

- 테스트 데이터셋 분류 정확도: 약 91.6%
- ViT 모델과 비교했을 때 F1 성능 지표는 거의 동일
- 검증 손실값은 0.4357에서 0.2416으로 감소

- **혼돈 행렬 분석**
  - 스윈 트랜스포머 모델이 ViT 모델보다 셔츠 이미지를 더 높은 확률로 예측함
  - 티셔츠, 코드로 오분류하는 문제가 완화된 것을 확인 가능
  - 또한 트랜스포머 모델의 수렴 속도가 더 빠른 것을 확인할 수 있음

10.3 CvT

- **CvT(Convolutional Vision Transformer):** 합성곱 신경망에서 사용하는 계층형 구조(Hierarchical Architecture)를 ViT에 적용한 모델
- CvT의 등장 배경
  - ViT: 모든 패치에 대해 동일한 크기와 간격을 사용하므로 이미지의 물체 크기나 해상도를 계층적으로 학습하기 어려움
  - Swin Transformer: 블록마다 다른 로컬 윈도 크기를 사용해 이미지의 계층적 특징을 학습
  - CvT: 합성곱 신경망에서 사용하는 계층적 구조를 ViT에 적용해 이미지의 지역 특징(Local Feature)과 전역 특징(Global Feature)을 함께 학습
    - 합성곱 계층의 지역 특징 추출하고, ViT의 셀프 어텐션 계층으로 전역 특징을 결합해 이미지를 처리함

    → CvT가 비교적 적은 수의 매개변수로 높은 성능을 보임

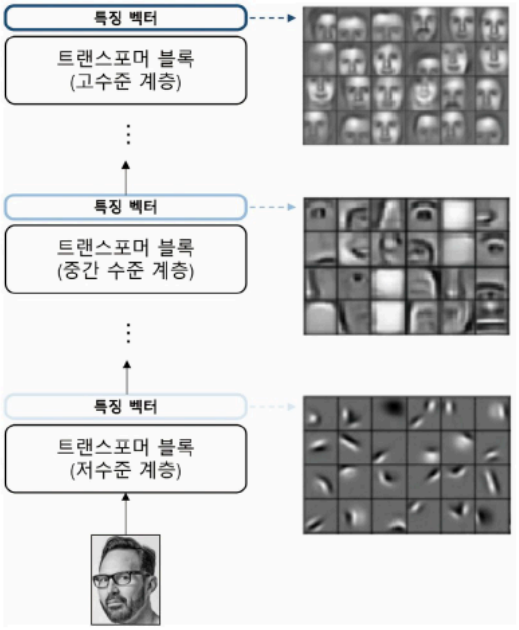


그림 10.14 CvT 모델의 계층적 특징 구조 예시

- CvT의 계층적 특징(얼굴 이미지 예시)
  1. 저수준(Low-Level): 선 또는 점에 대한 특징
  2. 중간 수준(Mid-Level): 눈이나 귀에 대한 특징
  3. 고수준(High-Level): 전체 얼굴 구조에 대한 특징

  → 계층 깊어질수록 더 복잡한 이미지 특징 학습. CvT의 계층형 구조는 물체의 크기나 해상도를 더 잘 표현할 수 있음
- ViT와 CvT의 이미지 처리 방식
  - ViT: 이미지를 겹치지 않는 패치로 분할, 이를 트랜스포머 모델의 입력으로 사용
    - 패치를 겹치지 않게 처리해 모델이 이미지의 전체 정보를 학습
    - 패치 임베딩과 선형 임베딩 사용
  - CvT: 이미지를 처리하기 전에 합성곱 연산을 통해 특징 맵 추출 → 유의미한 패턴 추출하고 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력으로 사용함
    - 입력되는 패치들은 겹쳐 학습함
    - 합성곱 연상 통해 이미지 학습 → 토큰에 대한 위치 임베딩 적용 X도 모델 성능 저하 없이 다양한 해상도를 가진 이미지 처리 가능

표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

10.3.1 합성곱 토큰 임베딩

- **Convolutional Token Embedding:** 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩
  - ViT에 계층적인 합성곱 신경망 성질을 추가하기 위해 사용
- **기존 Transformer**
  - 이미지 특징 추출하기 위해 9개 이미지 패치 사용
  - 선형 임베딩 통해 쿼리, 키, 값 임베딩으로 변환
  - 이미지의 근본적인 2D 구조를 잃게 됨
- **CvT**
  - 이미지의 2D 구조 보존하면서도 Self-Attention 계산
  - 이미지 패치에 2D 합성곱 임베딩을 적용하여 쿼리, 키, 값 생성
  - 변환된 임베딩을 이용해 Self-Attention 수행

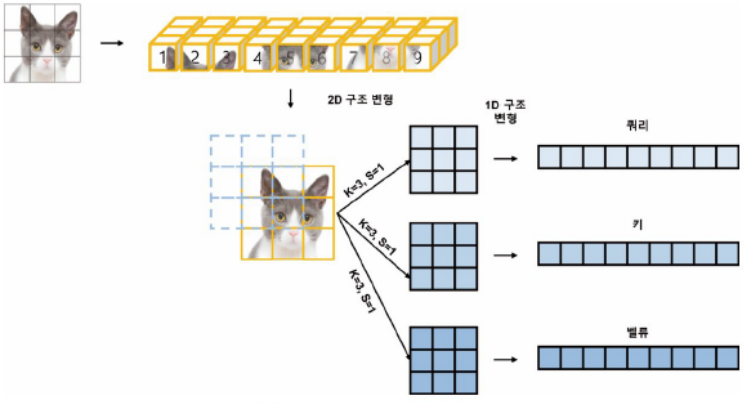


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 그림 10.15 합성곱 토큰 임베딩
  - 이미지에 3x3 Kernel Size, Stride 1의 합성곱 적용
  - 각 패치에서 Q, K, V에 대한 합성곱 연산 수행
  - 1차원 구조로 변환하여 Self-Attention 계산
- 특징

- 지역 특징 학습 가능
- 시퀀스 길이를 줄이는 동시에 토큰 특징의 차원을 증가시킴
- (합성곱 모델에서 수행되는 방식처럼) 다운 샘플링과 특징 맵의 수를 늘리는 방식 사용

10.3.2 어텐션에 대한 합성곱 임베딩

- **Convolutional Projection for Attention**: 2D로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산을 적용하여 성능 저하 및 계산 복잡도를 낮추는 연산
- **Q, K, V 차원 축소**
  - 일반적으로 Q, K, V의 임베딩 차원을 동일하게 설정
  - CvT는 매개변수 수를 줄이기 위해 Q, K, V의 차원을 축소
  - 그림 10.16에서는 **2D 합성곱 기반의 축소된 어텐션 임베딩(Squeezed Convolutional Projection)**을 보여줌

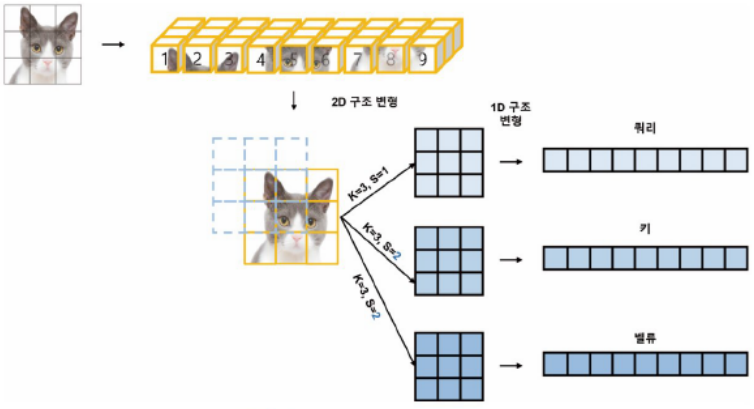


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

1. Query: 3×3 Kernel, Stride 1 → 9개 패치
2. Key: 3×3 Kernel, Stride 2 → 4개 패치
3. Value: 3×3 Kernel, Stride 2 → 4개 패치

- **축소된 셀프 어텐션**
  - Query: 9×1
  - Key: 1×4
  - Attention Map: 9×4
  - Value: 4×1

- 최종 Self-Attention Vector: 9×1

  → Key와 Value의 패치 개수가 줄어들어 연산량 감소

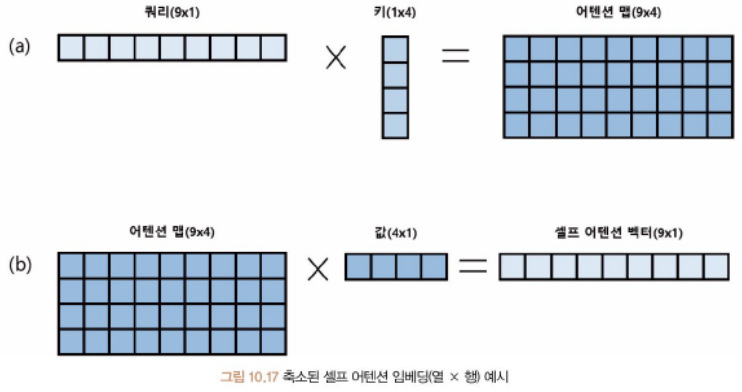


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

- 계층별 특징 변화
  - 트랜스포머 계층 깊어질수록 이미지 패치 크기 축소
  - 패치 사이의 관계를 학습하는 길이를 줄이는 대신 임베딩 벡터 차원 증가
  - 예시
    - 첫 번째 블록: 이미지 패치 크기 64x64, 임베딩 차원 128
    - 마지막 블록: 이미지 패치 크기 8x8, 임베딩 차원 512

  → 패치 크기가 작아질수록 임베딩 차원을 늘리는 방식으로 모델 구성

  ∴ 계산 복잡도를 감소시키면서 모델의 안정성 향상

10.3.3 모델 실습

- 앞선 ViT 실습 코드를 활용해 CvT 모델 미세 조정
- 허깅 페이스에서 제공하는 사전 학습된 `microsoft/cvt-21` 모델 사용
  - ImageNet-1k 데이터셋으로 학습
  - 입력 이미지 크기: 224x224
- FashionMNIST 데이터셋 사용

1. CvT 모델 이미지 데이터 전처리

- **ViT 및 Swin Transformer와의 차이**

- 기존: `height` , `width` 를 사용하여 이미지 크기 조정
- CvT: `shortest_edge` 를 사용하여 이미지 크기 조정

- **shortest_edge:** 이미지의 너비와 높이 중 더 작은 값
  - CvT는 입력 이미지의 가장 짧은 길이를 기준으로 정규화
  - 이미지 크기를 조정할 때 `shortest_edge` 사용

2. 사전 학습된 CvT 모델

- **CvT 모델 구조**
  - 크게 `cvt` , `layernorm` , `classifier` 로 구성
  - `cvt` 는 세 개의 스테이지(stage 0~2)로 구성
- **Swin Trasnformer와의 차이**
  1. CvT는 분류기 이전에 풀링 계층이 존재하지 않음
     - 이미 합성곱 계층을 활용해 지역 특징 정보와 전역 정보를 파악하기 때문
  2. 스테이지별 이미지 크기와 임베딩 차원 변화
     - 이미지 사이즈는 스테이지별로 4배, 8배, 16배 감소되며
     - 반비례하게 임베딩 차원이 커짐

3. CvT 모델의 스테이지 구조

- 예제 10.22에서는 첫 번째 스테이지 구조를 출력함
- 각 스테이지는 크게 `CvtEmbedding` 와 `CvtLayer` 로 구성됨

1. `CvtEmbedding`
   - 입력 이미지에 대한 2D 합성곱 연산과 계층 정규화로 이뤄져 있음
   - 이미지를 패치 단위로 잘라 임베딩하는 역할
   - 입력 이미지는 첫 번째 스테이지의 `CvtEmbedding` 에 적용됨 7x7 커널 크기, 4 간격, 2 패딩 크기로 합성곱 연산을 적용함
   - 224×224 크기의 이미지에 합성곱 연산 적용하면 56.25의 크기 반환됨
   - 소수점 발생하므로 버림 처리하면 56x56 크기의 텐서가 생성됨
   - 출력 채널의 크기가 64이므로 [N, 64, 56, 56]의 텐서가 반환됨

- 매개변수가 축소된 어텐션 임베딩 계산 위해 키와 값 임베딩의 간격을 1보다 큰 2로 설정해 차원 축소하는 것을 확인 가능

2. `CvtLayer`
    - 3,136(56×56)개의 이미지 패치에 대한 셀프 어텐션 수행됨

4. 셀프 어텐션 적용

- 셀프 어텐션 모듈에 이미지를 전달했을 때 셀프 어텐션의 결괏값을 보여줌
- **출력 결과**
    - 이미지 차원: [32, 3, 224, 224]
    - 패치 임베딩 차원: [32, 64, 56, 56]
    - 셀프 어텐션 입력 차원: [32, 3136, 64]
    - 셀프 어텐션 출력 차원: [32, 3136, 64]
- 합성곱 계층 통해 계산된 패치 임베딩은 가로와 세로 크기로 재정렬해 CvtLayer에 입력됨
- CvtLayer의 `attention.attention` 계층을 통해 셀프 어텐션 수행됨

  → 모듈의 입력 차원과 출력 차원의 형태가 동일함

5. CvT 모델 학습

- 지금까지 CvT 모델의 구조 알아봄
- 이제 FashionMNIST 데이터세트를 활용해 CvT 모델을 미세 조정해보자
- 미세 조정 학습 과정
    - 훈련자 클래스(Trainer)를 통해 CvT 모델의 학습 수행함
    - 변경된 곳은 사전 학습된 모델, 전처리 과정, 출력 디렉터
    - 학습 완료되면 CvT-FashionMNIST 폴더에 체크포인트마다 학습 결과, 성능 지표, 하이퍼파라미터 등이 저장됨

표 10.5 CvT 학습 결과

Epoch	Training Loss	Validation Loss	F1
1	0.7737	0.3269	0.8914
2	0.7008	0.2596	0.9172
3	0.6438	0.2460	0.9173

- 학습 결과
  - CvT 모델은 91.3%의 분류 정확도를 가짐
  - 스윈 트랜스포머 모델과 CvT 모델의 F1 점수 결과를 비교했을 때 0.9159에서 0.9173으로 소폭 개선됨
- **혼동 행렬**
  - ViT 모델과 비교하면 셔츠 카테고리에 대한 성능 개선됨
  - 스윈 트랜스포머와 비슷한 성능을 보임

표 10.6 비전 트랜스포머 모델 비교

	ViT	스윈 트랜스포머	CvT
모델 크기	327.325 MB	105.227 MB	120.791 MB
F1-점수	0.9231	0.9159	0.9173
초당 처리 수	29.636 eval/s	58.048 eval/s	44.778 eval/s

→ FashionMNIST 데이터세트에서 비전 트랜스포머 계열 모델은 작은 학습 데이터세트에서도 높은 정확도와 우수한 성능을 보임

→ 가장 일반화 성능이 좋았던 모델 = CvT 모델 / 모델 크기 대비 가장 우수한 성능 = Swin Transformer 모델 / F1- 점수가 가장 높은 모델 = ViT 모델

∴비전 트랜스포머 모델은 분류 문제에 유용하며, 해결하려는 문제와 사용 환경에 따라 적절한 모델을 선택해야 함